

Отчёт по лабораторной работе №2

Простые сети в GNS3. Анализ трафика

Цыкунова Екатерина Михайловна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Моделирование простейшей сети на базе коммутатора в GNS3 . .	6
2.2	Анализ трафика в GNS3 посредством Wireshark	10
2.3	Моделирование простейшей сети на базе маршрутизатора FRR в GNS3	15
2.4	Моделирование простейшей сети на базе маршрутизатора VyOS в GNS3	19
3	Вывод	25

Список иллюстраций

2.1	Топология простейшей сети на базе коммутатора Ethernet	6
2.2	Просмотр доступных команд VPCS	7
2.3	Настройка IP-адреса PC1 и сохранение конфигурации	8
2.4	Настройка IP-адреса PC2 и сохранение конфигурации	9
2.5	Проверка соединения между PC1 и PC2	10
2.6	Анализ ARP-сообщений в Wireshark	11
2.7	Выполнение одного эхо-запроса в ICMP-режиме	12
2.8	Анализ ICMP Echo Request и Echo Reply в Wireshark	13
2.9	Выполнение одного запроса в UDP-режиме	13
2.10	Анализ UDP-запроса и ответа в Wireshark	14
2.11	Выполнение одного запроса в TCP-режиме	14
2.12	Анализ TCP-соединения в Wireshark	15
2.13	Настройка IP-адреса PC1 в сети с маршрутизатором FRR	16
2.14	Настройка имени и интерфейса eth0 маршрутизатора FRR	17
2.15	Проверка текущей конфигурации маршрутизатора FRR	17
2.16	Проверка состояния интерфейсов маршрутизатора FRR	18
2.17	Проверка соединения между PC1 и маршрутизатором FRR	18
2.18	Анализ ICMP-трафика между PC1 и маршрутизатором FRR	19
2.19	Топология простейшей сети на базе маршрутизатора VyOS	20
2.20	Проверка IP-конфигурации PC1 в сети с VyOS	20
2.21	Настройка маршрутизатора VyOS	21
2.22	Проверка конфигурации интерфейсов маршрутизатора VyOS	22
2.23	Проверка соединения между PC1 и маршрутизатором VyOS	23
2.24	Анализ ARP- и ICMP-трафика между PC1 и маршрутизатором VyOS	24

Список таблиц

1 Цель работы

Построение простейших моделей сети на базе коммутатора и маршрутизаторов FRR и VyOS в GNS3, анализ трафика посредством Wireshark.

2 Выполнение лабораторной работы

2.1 Моделирование простейшей сети на базе коммутатора в GNS3

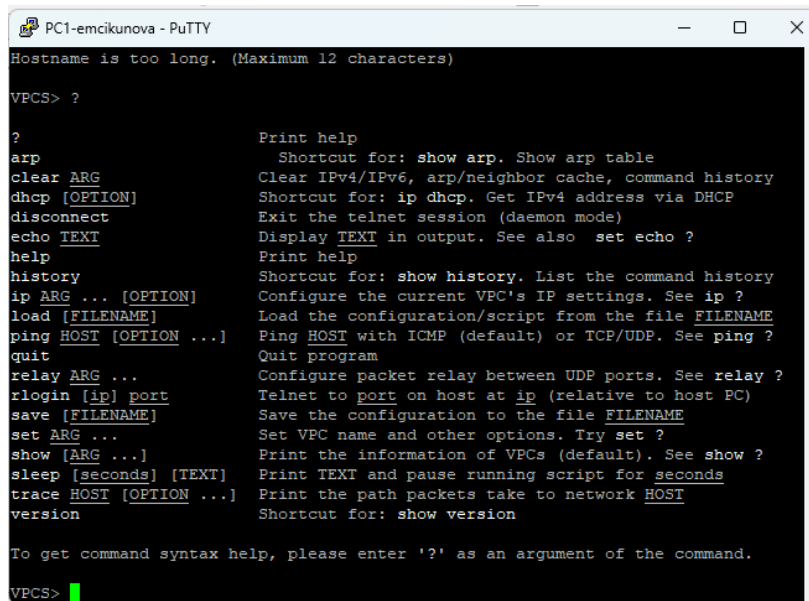
Запускаю GNS3 VM и GNS3 и создаю новый проект. В рабочей области размещаю коммутатор Ethernet и два оконечных устройства VPCS. Переименовываю устройства в соответствии с используемой учётной записью: PC1-emsikunova, PC2-emsikunova и msk-emsikunova-sw-01. Соединяю оба компьютера с коммутатором Ethernet (см. рис. Рисунок 2.1).



Рисунок 2.1: Топология простейшей сети на базе коммутатора Ethernet

Запускаю PC1-emsikunova и открываю его консоль. Для просмотра доступных команд VPCS ввожу команду ?. В терминале отображается список команд для настройки IP-адресации, проверки соединения, просмотра параметров и со-

хранения конфигурации (см. рис. Рисунок 2.2).



```
PC1-emcikunova - PuTTY
Hostname is too long. (Maximum 12 characters)
VPCS> ?

?          Print help
arp        Shortcut for: show arp. Show arp table
clear ARG  Clear IPv4/IPv6, arp/neighbor cache, command history
dhcp [OPTION] Shortcut for: ip dhcp. Get IPv4 address via DHCP
disconnect Exit the telnet session (daemon mode)
echo TEXT  Display TEXT in output. See also set echo ?
help       Print help
history     Shortcut for: show history. List the command history
ip ARG ... [OPTION] Configure the current VPC's IP settings. See ip ?
load [FILENAME] Load the configuration/script from the file FILENAME
ping HOST [OPTION ...] Ping HOST with ICMP (default) or TCP/UDP. See ping ?
quit       Quit program
relay ARG ... Configure packet relay between UDP ports. See relay ?
rlogin [ip] port Telnet to port on host at ip (relative to host PC)
save [FILENAME] Save the configuration to the file FILENAME
set ARG ... Set VPC name and other options. Try set ?
show [ARG ...] Print the information of VPCs (default). See show ?
sleep [seconds] [TEXT] Print TEXT and pause running script for seconds
trace HOST [OPTION ...] Print the path packets take to network HOST
version     Shortcut for: show version

To get command syntax help, please enter '?' as an argument of the command.
VPCS>
```

Рисунок 2.2: Просмотр доступных команд VPCS

Для первого оконечного устройства просматриваю синтаксис команды `ip`, после чего задаю IP-адрес `192.168.1.11/24` и шлюз `192.168.1.1` командой `ip 192.168.1.11/24 192.168.1.1`. Для сохранения параметров выполняю команду `save`. Конфигурация успешно сохраняется в файл `startup.vpc` (см. рис. Рисунок 2.3).

```
PC1-emcikunova - PuTTY

To get command syntax help, please enter '?' as an argument of the command.

VPCS> ip ?

ip ARG ... [OPTION]
Configure the current VPC's IP settings
ARG ...:
  address [mask] [gateway]
  address [gateway] [mask]
Set the VPC's ip, default gateway ip and network mask
Default IPv4 mask is /24, IPv6 is /64. Example:
ip 10.1.1.70/26 10.1.1.65 set the VPC's ip to 10.1.1.70,
the gateway to 10.1.1.65, the netmask to 255.255.255.192.
In tap mode, the ip of the tapx is the maximum host ID
of the subnet. In the example above the tapx ip would be
10.1.1.126
mask may be written as /26, 26 or 255.255.255.192
auto Attempt to obtain IPv6 address, mask and gateway using SLAAC
dhcp [OPTION] Attempt to obtain IPv4 address, mask, gateway, DNS via DHCP
  -d Show DHCP packet decode
  -r Renew DHCP lease
  -x Release DHCP lease
dns ip Set DNS server ip, delete if ip is '0'
dns6 ipv6 Set DNS server ipv6, delete if ipv6 is '0'
domain NAME Set local domain name to NAME

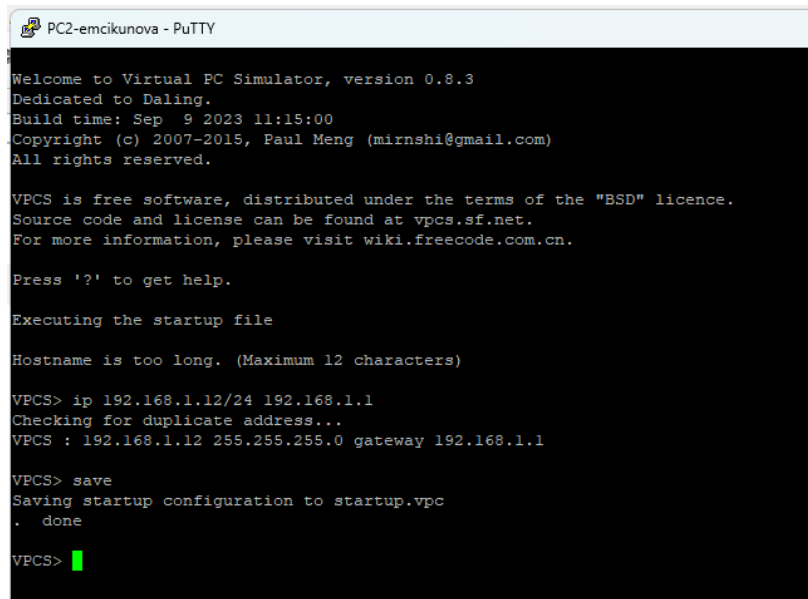
VPCS> ip 192.168.1.11/24 192.168.1.1
Checking for duplicate address...
VPCS : 192.168.1.11 255.255.255.0 gateway 192.168.1.1

VPCS> save
Saving startup configuration to startup.vpc
. done

VPCS>
```

Рисунок 2.3: Настройка IP-адреса PC1 и сохранение конфигурации

Аналогично настраиваю второе оконечное устройство PC2-emcikunova. Назначаю ему IP-адрес 192.168.1.12/24 со шлюзом 192.168.1.1 и сохраняю конфигурацию командой save (см. рис. Рисунок 2.4).



```
PC2-emcikunova - PuTTY

Welcome to Virtual PC Simulator, version 0.8.3
Dedicated to Daling.
Build time: Sep 9 2023 11:15:00
Copyright (c) 2007-2015, Paul Meng (mirnshi@gmail.com)
All rights reserved.

VPCS is free software, distributed under the terms of the "BSD" licence.
Source code and license can be found at vpcs.sf.net.
For more information, please visit wiki.freecode.com.cn.

Press '?' to get help.

Executing the startup file

Hostname is too long. (Maximum 12 characters)

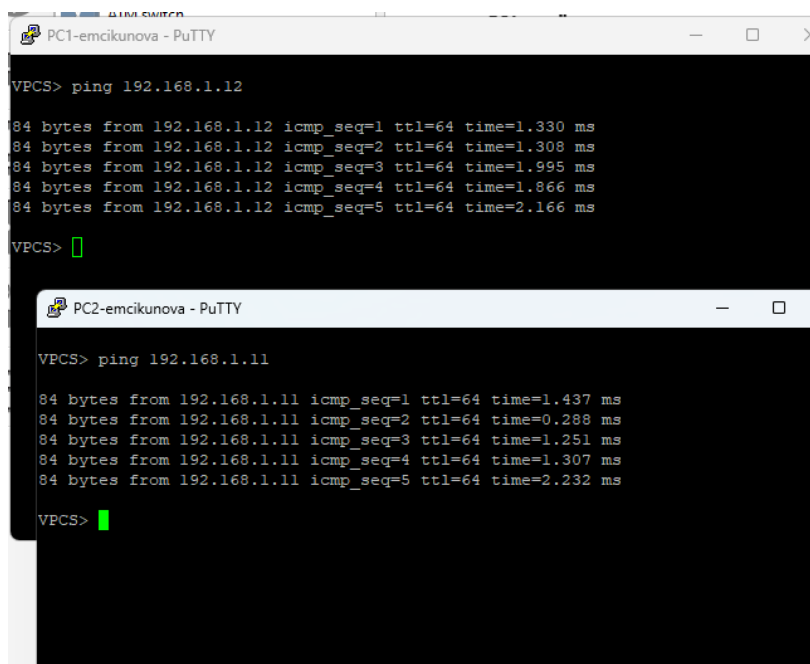
VPCS> ip 192.168.1.12/24 192.168.1.1
Checking for duplicate address...
VPCS : 192.168.1.12 255.255.255.0 gateway 192.168.1.1

VPCS> save
Saving startup configuration to startup.vpc
. done

VPCS> █
```

Рисунок 2.4: Настройка IP-адреса PC2 и сохранение конфигурации

Проверяю соединение между оконечными устройствами в обоих направлениях. С PC1-емcikunova выполняю ping 192.168.1.12, а с PC2-емcikunova — ping 192.168.1.11. Во всех случаях получаю ответы от удалённого узла, потери пакетов отсутствуют. Следовательно, устройства находятся в одной сети 192.168.1.0/24 и успешно обмениваются данными через Ethernet-коммутатор (см. рис. Рисунок 2.5).



The image shows two terminal windows from a GNS3 simulation. The top window, titled 'PC1-emcikunova - PuTTY', shows the command 'VPCS> ping 192.168.1.12' and its output: five successful ping responses from 192.168.1.12 with varying times (1.330 ms to 2.166 ms). The bottom window, titled 'PC2-emcikunova - PuTTY', shows the command 'VPCS> ping 192.168.1.11' and its output: five successful ping responses from 192.168.1.11 with varying times (1.437 ms to 2.232 ms).

```
PC1-emcikunova - PuTTY
VPCS> ping 192.168.1.12
84 bytes from 192.168.1.12 icmp_seq=1 ttl=64 time=1.330 ms
84 bytes from 192.168.1.12 icmp_seq=2 ttl=64 time=1.308 ms
84 bytes from 192.168.1.12 icmp_seq=3 ttl=64 time=1.995 ms
84 bytes from 192.168.1.12 icmp_seq=4 ttl=64 time=1.866 ms
84 bytes from 192.168.1.12 icmp_seq=5 ttl=64 time=2.166 ms
VPCS>

PC2-emcikunova - PuTTY
VPCS> ping 192.168.1.11
84 bytes from 192.168.1.11 icmp_seq=1 ttl=64 time=1.437 ms
84 bytes from 192.168.1.11 icmp_seq=2 ttl=64 time=0.288 ms
84 bytes from 192.168.1.11 icmp_seq=3 ttl=64 time=1.251 ms
84 bytes from 192.168.1.11 icmp_seq=4 ttl=64 time=1.307 ms
84 bytes from 192.168.1.11 icmp_seq=5 ttl=64 time=2.232 ms
VPCS>
```

Рисунок 2.5: Проверка соединения между PC1 и PC2

2.2 Анализ трафика в GNS3 посредством Wireshark

Запускаю захват трафика на соединении между PC1-emcikunova и коммутатором. После запуска устройств в Wireshark появляются ARP-пакеты. В захваченном трафике видны широковещательные Gratuitous ARP-запросы для адресов 192.168.1.11 и 192.168.1.12. Ethernet-кадры передаются на широковещательный MAC-адрес ff:ff:ff:ff:ff:ff. Например, устройство с адресом 192.168.1.12 формирует Gratuitous ARP, в котором IP-адрес отправителя и запрашиваемый IP-адрес равны 192.168.1.12. Подобные сообщения позволяют объявить используемый IP-адрес в локальной сети и обнаружить возможное дублирование адреса (см. рис. Рисунок 2.6).

[illegible]

Рисунок 2.6: Анализ ARP-сообщений в Wireshark

В терминале PC2-emsikunova просматриваю параметры команды ping. VPCS позволяет выполнять проверку в трёх режимах: ICMP (-1), UDP (-2) и TCP (-3). Для отправки одного ICMP-запроса к PC1-emsikunova выполняю команду ping 192.168.1.11 -c 1 -1. Получаю успешный ответ от узла 192.168.1.11 (см. рис. Рисунок 2.7).

```

VPCS> ping ?
ping HOST [OPTION ...]
Ping the network HOST. HOST can be an ip address or name
Options:
  -l          ICMP mode, default
  -2          UDP mode
  -3          TCP mode
  -c count    Packet count, default 5
  -D          Set the Don't Fragment bit
  -f FLAG     Top header FLAG |C|E|U|A|P|R|S|F|
               bits |7 6 5 4 3 2 1 0|
  -i ms       Wait ms milliseconds between sending each packet
  -l size     Data size
  -P protocol Use IP protocol in ping packets
               1 - ICMP (default), 17 - UDP, 6 - TCP
  -p port     Destination port
  -s port     Source port
  -T ttl      Set ttl, default 64
  -t         Send packets until interrupted by Ctrl+C
  -w ms       Wait ms milliseconds to receive the response

Notes: 1. Using names requires DNS to be set.
       2. Use Ctrl+C to stop the command.

VPCS> ping 192.168.1.11 -c 1 -l
84 bytes from 192.168.1.11 icmp_seq=1 ttl=64 time=2.466 ms
VPCS>

```

Рисунок 2.7: Выполнение одного эхо-запроса в ICMP-режиме

Анализирую ICMP-обмен в Wireshark. Перед непосредственной отправкой ICMP-пакета узел 192.168.1.12 выполняет разрешение MAC-адреса узла 192.168.1.11 с помощью ARP-запроса Who has 192.168.1.11? Tell 192.168.1.12. В ответ узел 192.168.1.11 сообщает свой MAC-адрес 00:50:79:66:68:00. После получения MAC-адреса передаётся ICMP Echo (ping) request от 192.168.1.12 к 192.168.1.11, а затем ICMP Echo (ping) reply в обратном направлении. Значение TTL в захваченных пакетах равно 64 (см. рис. Рисунок 2.8).

No.	Time	Source	Destination	Protocol	Length	Info
4	0.051087	Private_66:68:01	Broadcast	ARP	64	Gratuitous ARP for 192.168.1.12 (Request)
5	1.050858	Private_66:68:01	Broadcast	ARP	64	Gratuitous ARP for 192.168.1.12 (Request)
6	1.050890	Private_66:68:00	Broadcast	ARP	64	Gratuitous ARP for 192.168.1.11 (Request)
7	2.051859	Private_66:68:00	Broadcast	ARP	64	Gratuitous ARP for 192.168.1.11 (Request)
8	2.052898	Private_66:68:01	Broadcast	ARP	64	Gratuitous ARP for 192.168.1.12 (Request)
9	128.820135	Private_66:68:01	Broadcast	ARP	64	Who has 192.168.1.11? Tell 192.168.1.12
10	128.821401	Private_66:68:00	Private_66:68:01	ARP	64	192.168.1.11 is at 00:50:79:66:68:00
11	128.823979	192.168.1.12	192.168.1.11	ICMP	98	Echo (ping) request id=0x677d, seq=1/256, ttl=64 (reply in 12)
12	128.825115	192.168.1.11	192.168.1.12	ICMP	98	Echo (ping) reply id=0x677d, seq=1/256, ttl=64 (request in 11)

Character encoding: ASCII (0)	0000	00 50 79 66 68 00 00 50 79 66 68 01 00 00 45 00	Pyfh P yfh...E
[Coloring Rule Name: ICMP]	0010	00 54 7d 67 00 00 40 01 79 da c0 a3 01 0c c0 a3	T]g @ y.....
[Coloring Rule String: icmp icmpv6]	0020	01 0b 08 00 b8 8d 67 7d 00 01 08 09 0a 0b 0c 0d	g}.....
▼ Ethernet II, Src: Private_66:68:01 (00:50:79:66:68:01), Dst: Private_66:68:00 (00:50:79:66:68:00)	0030	0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d	
> Destination: Private_66:68:00 (00:50:79:66:68:00)	0040	1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d	..!""\$%&'()*+,-
> Source: Private_66:68:01 (00:50:79:66:68:01)	0050	2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d	./012345 6789;<=
> Type: IPv4 (0x0800)	0060	3e 3f	>?
[Stream index: 4]			
▼ Internet Protocol Version 4, Src: 192.168.1.12, Dst: 192.168.1.11			
0100 = Version: 4			
.... 0101 = Header Length: 20 bytes (5)			
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)			
Total Length: 84			
Identification: 0x7d67 (32103)			
> 0000 = Flags: 0x0			
...0 0000 0000 0000 = Fragment Offset: 0			
Time to live: 64			
Protocol: ICMP (1)			
Header Checksum: 0x79da [validation disabled]			
[Header checksum status: Unverified]			
Source Address: 192.168.1.12			
Destination Address: 192.168.1.11			
[Stream index: 0]			
▼ Internet Control Message Protocol			
Type: Echo (ping) request (0)			
Code: 0			
Checksum: 0xb88d [correct]			
[Checksum Status: Good]			
Identifier (BE): 26493 (0x677d)			
Identifier (LE): 32103 (0x7d67)			
Sequence Number (BE): 1 (0x0001)			
Sequence Number (LE): 256 (0x0100)			
[Response frame: 12]			
> Data (56 bytes)			

Рисунок 2.8: Анализ ICMP Echo Request и Echo Reply в Wireshark

Далее проверяю работу команды ping в UDP-режиме. На PC2-емсикупова выполняю ping 192.168.1.11 -c 1 -2. Получаю успешный ответ от узла 192.168.1.11 с указанием udp_seq=1 (см. рис. Рисунок 2.9).

```

VPCS>
VPCS>
VPCS> ping 192.168.1.11 -c 1 -2

84 bytes from 192.168.1.11 udp_seq=1 ttl=64 time=1.360 ms

VPCS>

```

Рисунок 2.9: Выполнение одного запроса в UDP-режиме

В Wireshark наблюдаю UDP-пакет от 192.168.1.12 к 192.168.1.11. В заголовке IPv4 указано значение Protocol: UDP (17). В UDP-заголовке видно, что запрос направляется на порт 7, соответствующий службе Echo. После запроса от 192.168.1.12 принимается ответ от 192.168.1.11. В отличие от TCP, протокол UDP не устанавливает предварительное соединение между узлами (см. рис. Рисунок 2.10).

No.	Time	Source	Destination	Protocol	Length	Info
6	1.050890	Private_66:68:00	Broadcast	ARP	64	Gratuitous ARP for 192.168.1.11 (Request)
7	2.051859	Private_66:68:00	Broadcast	ARP	64	Gratuitous ARP for 192.168.1.11 (Request)
8	2.052898	Private_66:68:01	Broadcast	ARP	64	Gratuitous ARP for 192.168.1.12 (Request)
9	128.820135	Private_66:68:01	Broadcast	ARP	64	Who has 192.168.1.11? Tell 192.168.1.12
10	128.821481	Private_66:68:00	Private_66:68:01	ARP	64	192.168.1.11 is at 00:50:79:66:68:00
11	128.823979	192.168.1.12	192.168.1.11	ICMP	98	Echo (ping) request id=0x677d, seq=1/256, ttl=64 (reply in 12)
12	128.825115	192.168.1.11	192.168.1.12	ICMP	98	Echo (ping) reply id=0x677d, seq=1/256, ttl=64 (request in 11)
13	217.788118	192.168.1.12	192.168.1.11	ECHO	98	Request
14	217.788812	192.168.1.11	192.168.1.12	ECHO	98	Response

Character encoding: ASCII (0) [Coloring Rule Name: UDP] [Coloring Rule String: udp] Ethernet II, Src: Private_66:68:01 (00:50:79:66:68:01), Dst: Private_66:68:00 (00:50:79:66:68:00) > Destination: Private_66:68:00 (00:50:79:66:68:00) > Source: Private_66:68:01 (00:50:79:66:68:01) Type: IPv4 (0x0800) [Stream index: 4] Internet Protocol Version 4, Src: 192.168.1.12, Dst: 192.168.1.11 0100 = Version: 4 0101 = Header Length: 20 bytes (5) > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT) Total Length: 84 Identification: 0x7dc0 (32192) > 0000 = Flags: 0x0 ...0 0000 0000 0000 = Fragment Offset: 0 Time to Live: 64 Protocol: UDP (17) Header Checksum: 0x7971 [validation disabled] [Header checksum status: Unverified] Source Address: 192.168.1.12 Destination Address: 192.168.1.11 [Stream index: 0] User Datagram Protocol, Src Port: 2596, Dst Port: 7 Source Port: 2596 Destination Port: 7 Length: 64 Checksum: 0xd650 [unverified] [Checksum Status: Unverified] [Stream index: 0] [Stream Packet Number: 1] [Timestamps] > UDP payload (56 bytes) > Echo	0000 00 50 79 66 68 00 00 50 79 66 68 01 00 00 45 00 Pyfh..P yfh...E 0010 00 54 7d c0 00 00 40 11 79 71 c0 a0 01 0c c0 a0 .T}...@ yq ... 0020 01 0b 0a 24 00 07 00 40 d6 50 00 50 79 66 68 01 ...\$...@ P Pyfh.. 0030 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 0040 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d ...!"#\$%&'()*+,- 0050 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d ./012345 6789;<= 0060 3e 3f >?
--	--

Рисунок 2.10: Анализ UDP-запроса и ответа в Wireshark

После этого выполняю один запрос в TCP-режиме командой `ping 192.168.1.11 -c 1 -3`. В терминале отображаются этапы Connect, SendData и Close, что соответствует установлению TCP-соединения, передаче данных и последующему закрытию соединения (см. рис. Рисунок 2.11).

```

VPCS>
VPCS> ping 192.168.1.11 -c 1 -3

Connect  7@192.168.1.11 seq=1 ttl=64 time=4.564 ms
SendData 7@192.168.1.11 seq=1 ttl=64 time=3.076 ms
Close    7@192.168.1.11 seq=1 ttl=64 time=3.044 ms

VPCS>

```

Рисунок 2.11: Выполнение одного запроса в TCP-режиме

Анализирую TCP-обмен в Wireshark. В начале соединения наблюдается стандартная последовательность установления TCP-соединения: пакет SYN от 192.168.1.12 к 192.168.1.11, ответ SYN, ACK и завершающий пакет ACK. Соединение устанавливается с TCP-портом назначения 7. После установки соединения передаётся Echo-запрос, принимается ответ, после чего начина-

ется завершение TCP-сессии с использованием флагов FIN и ACK. В отличие от UDP, для TCP требуется предварительное установление соединения и его корректное завершение (см. рис. Рисунок 2.12).

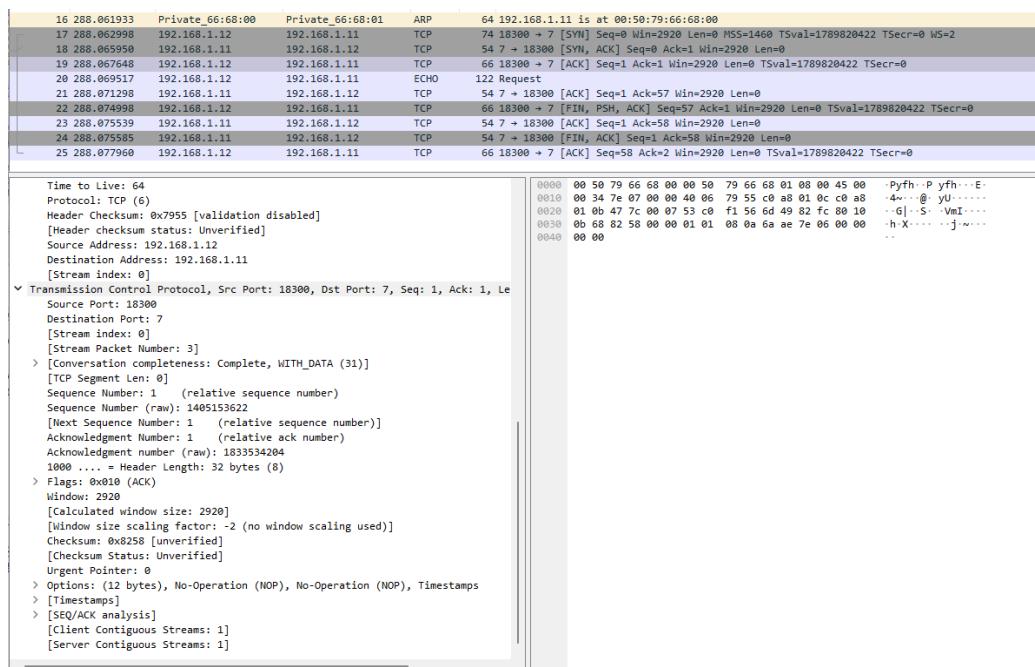


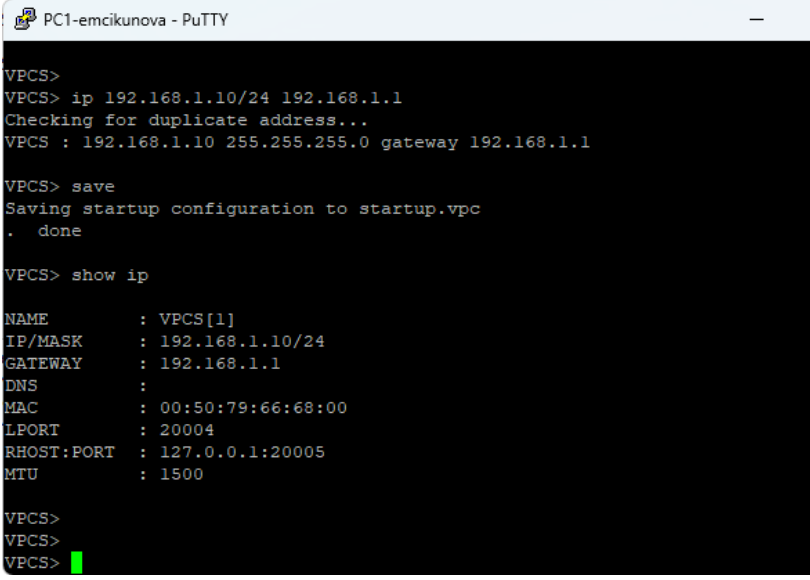
Рисунок 2.12: Анализ TCP-соединения в Wireshark

После завершения анализа останавливаю захват пакетов в Wireshark.

2.3 Моделирование простейшей сети на базе маршрутизатора FRR в GNS3

Создаю проект с маршрутизатором FRR, коммутатором Ethernet и одним оконечным устройством VPCS. Устройства переименовываю в соответствии с принятой схемой именования: маршрутизатор — msk-емсикунова-gw-01, коммутатор — msk-емсикунова-sw-01, оконечное устройство — PC1-емсикунова. На соединении между коммутатором и маршрутизатором включаю захват трафика.

На PC1-емсикунова задаю адрес 192.168.1.10/24 и шлюз 192.168.1.1 командой `ip 192.168.1.10/24 192.168.1.1`. Сохраняю настройки командой `save` и проверяю их командой `show ip`. В выводе отображаются IP-адрес 192.168.1.10/24 и шлюз 192.168.1.1 (см. рис. Рисунок 2.13).



```
PC1-емсикунова - PuTTY

VPCS>
VPCS> ip 192.168.1.10/24 192.168.1.1
Checking for duplicate address...
VPCS : 192.168.1.10 255.255.255.0 gateway 192.168.1.1

VPCS> save
Saving startup configuration to startup.vpc
. done

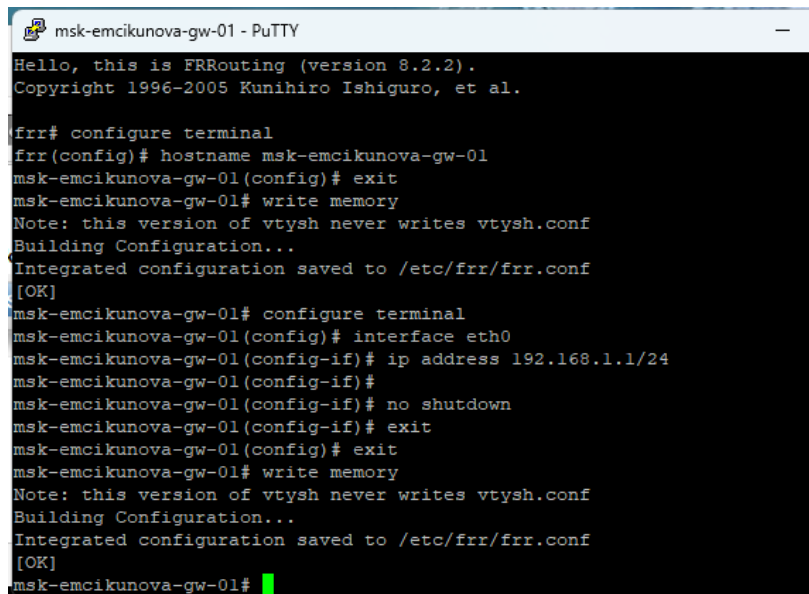
VPCS> show ip

NAME       : VPCS[1]
IP/MASK    : 192.168.1.10/24
GATEWAY    : 192.168.1.1
DNS        :
MAC        : 00:50:79:66:68:00
LPORT      : 20004
RHOST:PORT : 127.0.0.1:20005
MTU        : 1500

VPCS>
VPCS>
VPCS>
```

Рисунок 2.13: Настройка IP-адреса PC1 в сети с маршрутизатором FRR

Перехожу к настройке маршрутизатора FRR. В режиме конфигурирования изменяю имя устройства на `msk-емсикунова-gw-01` и сохраняю конфигурацию командой `write memory`. Затем перехожу к интерфейсу `eth0`, назначаю ему адрес 192.168.1.1/24 и активирую интерфейс командой `no shutdown`. После выхода из режима конфигурирования повторно сохраняю настройки. FRR подтверждает успешное сохранение конфигурации сообщением [OK] (см. рис. Рисунок 2.14).

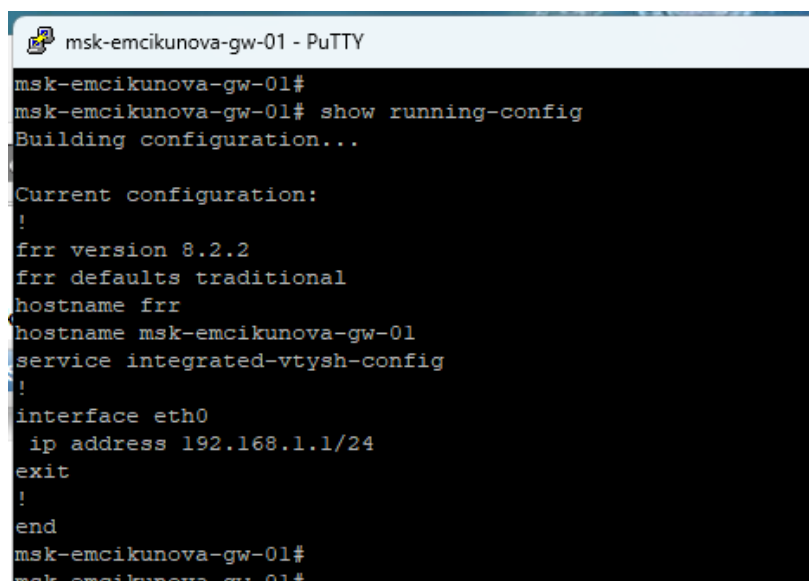


```
msk-emcikunova-gw-01 - PuTTY
Hello, this is FRRouting (version 8.2.2).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

frr# configure terminal
frr(config)# hostname msk-emcikunova-gw-01
msk-emcikunova-gw-01(config)# exit
msk-emcikunova-gw-01# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
msk-emcikunova-gw-01# configure terminal
msk-emcikunova-gw-01(config)# interface eth0
msk-emcikunova-gw-01(config-if)# ip address 192.168.1.1/24
msk-emcikunova-gw-01(config-if)#
msk-emcikunova-gw-01(config-if)# no shutdown
msk-emcikunova-gw-01(config-if)# exit
msk-emcikunova-gw-01(config)# exit
msk-emcikunova-gw-01# write memory
Note: this version of vtysh never writes vtysh.conf
Building Configuration...
Integrated configuration saved to /etc/frr/frr.conf
[OK]
msk-emcikunova-gw-01#
```

Рисунок 2.14: Настройка имени и интерфейса eth0 маршрутизатора FRR

Проверяю текущую конфигурацию маршрутизатора командой `show running-config`. В конфигурации присутствует имя `msk-emcikunova-gw-01`, а для интерфейса `eth0` указан адрес `192.168.1.1/24` (см. рис. Рисунок 2.15).



```
msk-emcikunova-gw-01 - PuTTY
msk-emcikunova-gw-01#
msk-emcikunova-gw-01# show running-config
Building configuration...

Current configuration:
!
frr version 8.2.2
frr defaults traditional
hostname frr
hostname msk-emcikunova-gw-01
service integrated-vtysh-config
!
interface eth0
 ip address 192.168.1.1/24
exit
!
end
msk-emcikunova-gw-01#
msk-emcikunova-gw-01#
```

Рисунок 2.15: Проверка текущей конфигурации маршрутизатора FRR

Дополнительно выполняю команду `show interface brief`. Интерфейс `eth0`

находится в состоянии up, принадлежит VRF default и использует адрес 192.168.1.1/24. Остальные физические Ethernet-интерфейсы в данной топологии не используются и находятся в состоянии down (см. рис. Рисунок 2.16).

```
mik-emsikunova-gw-01# show interface brief
Interface      Status  VRF      Addresses
-----
eth0            up      default  192.168.1.1/24
eth1            down    default
eth2            down    default
eth3            down    default
eth4            down    default
eth5            down    default
eth6            down    default
eth7            down    default
lo              up      default
pimreg          up      default
mik-emsikunova-gw-01#
```

Рисунок 2.16: Проверка состояния интерфейсов маршрутизатора FRR

С узла PC1-emsikunova проверяю доступность маршрутизатора командой ping 192.168.1.1. Получаю пять ответов от адреса 192.168.1.1 с TTL 64, что подтверждает корректность IP-настроек и работоспособность соединения между VPCS и маршрутизатором FRR (см. рис. Рисунок 2.17).

```
PC1-emsikunova - PuTTY
VPCS>
VPCS> ping 192.168.1.1

64 bytes from 192.168.1.1 icmp_seq=1 ttl=64 time=3.202 ms
64 bytes from 192.168.1.1 icmp_seq=2 ttl=64 time=2.262 ms
64 bytes from 192.168.1.1 icmp_seq=3 ttl=64 time=2.445 ms
64 bytes from 192.168.1.1 icmp_seq=4 ttl=64 time=2.618 ms
64 bytes from 192.168.1.1 icmp_seq=5 ttl=64 time=2.766 ms

VPCS>
VPCS>
VPCS>
```

Рисунок 2.17: Проверка соединения между PC1 и маршрутизатором FRR

Анализирую полученный трафик в Wireshark. В захвате присутствуют пары ICMP Echo (ping) request и Echo (ping) reply. Запросы передаются от 192.168.1.10 к интерфейсу маршрутизатора 192.168.1.1, а ответы — от 192.168.1.1 к 192.168.1.10. Для каждого отправленного запроса присутствует соответствующий ответ, что подтверждает отсутствие потерь на

проверяемом участке сети (см. рис. Рисунок 2.18).

No.	Time	Source	Destination	Protocol	Length	Info
17	196.329890	192.168.1.10	192.168.1.1	ICMP	98	Echo (ping) request id=0x637f, seq=1/256, ttl=64 (reply in 18)
18	196.332605	192.168.1.1	192.168.1.10	ICMP	98	Echo (ping) reply id=0x637f, seq=1/256, ttl=64 (request in 17)
19	197.334213	192.168.1.10	192.168.1.1	ICMP	98	Echo (ping) request id=0x647f, seq=2/512, ttl=64 (reply in 20)
20	197.335685	192.168.1.1	192.168.1.10	ICMP	98	Echo (ping) reply id=0x647f, seq=2/512, ttl=64 (request in 19)
21	198.337422	192.168.1.10	192.168.1.1	ICMP	98	Echo (ping) request id=0x657f, seq=3/768, ttl=64 (reply in 22)
22	198.339032	192.168.1.1	192.168.1.10	ICMP	98	Echo (ping) reply id=0x657f, seq=3/768, ttl=64 (request in 21)
23	199.340439	192.168.1.10	192.168.1.1	ICMP	98	Echo (ping) request id=0x667f, seq=4/1024, ttl=64 (reply in 24)
24	199.342201	192.168.1.1	192.168.1.10	ICMP	98	Echo (ping) reply id=0x667f, seq=4/1024, ttl=64 (request in 23)
25	200.343956	192.168.1.10	192.168.1.1	ICMP	98	Echo (ping) request id=0x677f, seq=5/1280, ttl=64 (reply in 26)
26	200.345675	192.168.1.1	192.168.1.10	ICMP	98	Echo (ping) reply id=0x677f, seq=5/1280, ttl=64 (request in 25)

▼ Ethernet II, Src: 0c:0b:52:3f:00:00 (0c:0b:52:3f:00:00), Dst: Private_66:68:00

> Destination: Private_66:68:00 (00:50:79:66:68:00)

> Source: 0c:0b:52:3f:00:00 (0c:0b:52:3f:00:00)

Type: IPv4 (0x0800)

[Stream index: 4]

▼ Internet Protocol Version 4, Src: 192.168.1.1, Dst: 192.168.1.10

0100 = Version: 4

.... 0101 = Header Length: 20 bytes (5)

> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

Total Length: 84

Identification: 0x6fe5 (28645)

> 0000 = Flags: 0x0

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 64

Protocol: ICMP (1)

Header Checksum: 0x8768 [validation disabled]

[Header checksum status: Unverified]

Source Address: 192.168.1.1

Destination Address: 192.168.1.10

[Stream index: 0]

▼ Internet Control Message Protocol

Type: Echo (ping) reply (0)

Code: 0

Checksum: 0xc48b [correct]

[Checksum Status: Good]

Identifier (BE): 25471 (0x637f)

Identifier (LE): 32611 (0x7f63)

Sequence Number (BE): 1 (0x0001)

Sequence Number (LE): 256 (0x0100)

[Request frame: 17]

[Response time: 2,715 ms]

> Data (56 bytes)

0000 00 50 79 66 68 00 0c 0b 52 3f 00 00 08 00 45 00 ·Pyfh... R?....E
0010 00 54 6f e5 00 00 40 01 87 68 c0 a8 01 01 c0 a8 ·To...@:·h.....
0020 01 0a 00 00 c4 8b 63 7f 00 01 08 09 0a 0b 0c 0dc:·h.....
0030 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d
0040 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d ····!"#\$%&'()*+,-
0050 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d ····./012345 6789;<=
0060 3e 3f ····?>

Рисунок 2.18: Анализ ICMP-трафика между PC1 и маршрутизатором FRR

После проверки останавливаю захват пакетов и все устройства проекта.

2.4 Моделирование простейшей сети на базе маршрутизатора VyOS в GNS3

Создаю топологию, состоящую из PC1-емсикunova, коммутатора msk-емсикunova-sw-01 и маршрутизатора VyOS msk-емсикunova-gw-01. Соединяю устройства последовательно через коммутатор. На соединении между коммутатором и маршрутизатором запускаю захват трафика, о чём свидетельствует значок анализатора на линии связи (см. рис. Рисунок 2.19).

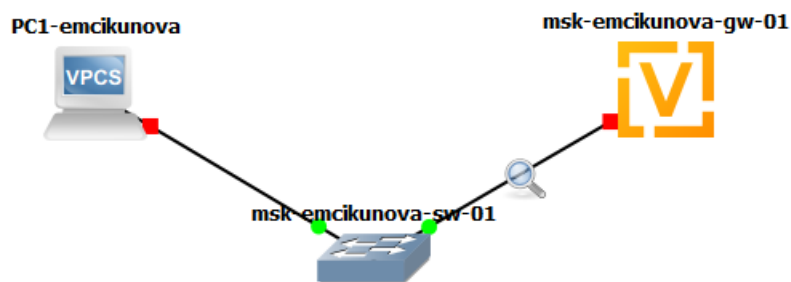


Рисунок 2.19: Топология простейшей сети на базе маршрутизатора VyOS

Запускаю оконечное устройство и проверяю его IP-конфигурацию командой `show ip`. Узлу назначен адрес 192.168.1.10/24, в качестве шлюза используется адрес 192.168.1.1 (см. рис. Рисунок 2.20).

```

PC1-emcikunova - PuTTY
VPCS is free software, distributed under the terms of the "BSD" licence.
Source code and license can be found at vpcs.sf.net.
For more information, please visit wiki.freecode.com.cn.

Press '?' to get help.

Executing the startup file

Checking for duplicate address...
VPCS : 192.168.1.10 255.255.255.0 gateway 192.168.1.1

VPCS> show ip

NAME       : VPCS[1]
IP/MASK    : 192.168.1.10/24
GATEWAY    : 192.168.1.1
DNS        :
MAC        : 00:50:79:66:68:00
LPORT      : 20004
RHOST:PORT : 127.0.0.1:20005
MTU        : 1500

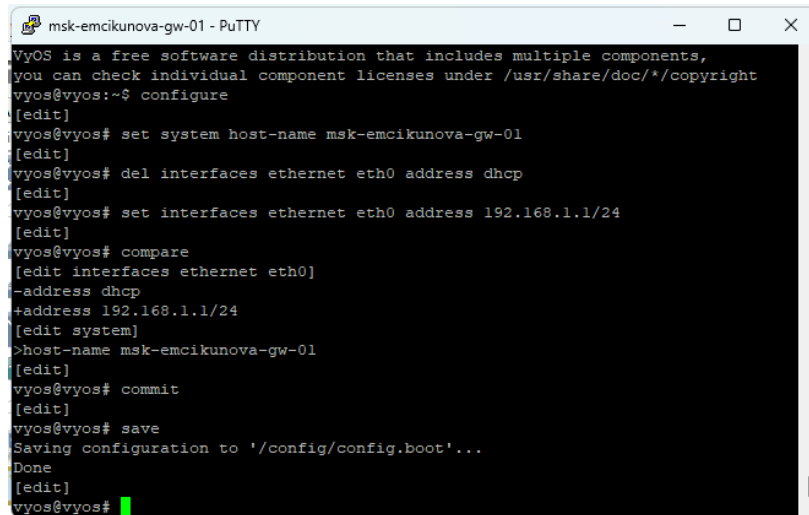
VPCS>

```

Рисунок 2.20: Проверка IP-конфигурации PC1 в сети с VyOS

Перехожу к настройке VyOS. Вхожу в режим конфигурирования командой `configure`, после чего задаю имя маршрутизатора командой `set system host-name msk-emcikunova-gw-01`. Удаляю получение адреса по DHCP с интерфейса `eth0` командой `del interfaces ethernet eth0 address dhcp` и назначаю статический адрес командой `set interfaces ethernet eth0 address 192.168.1.1/24`.

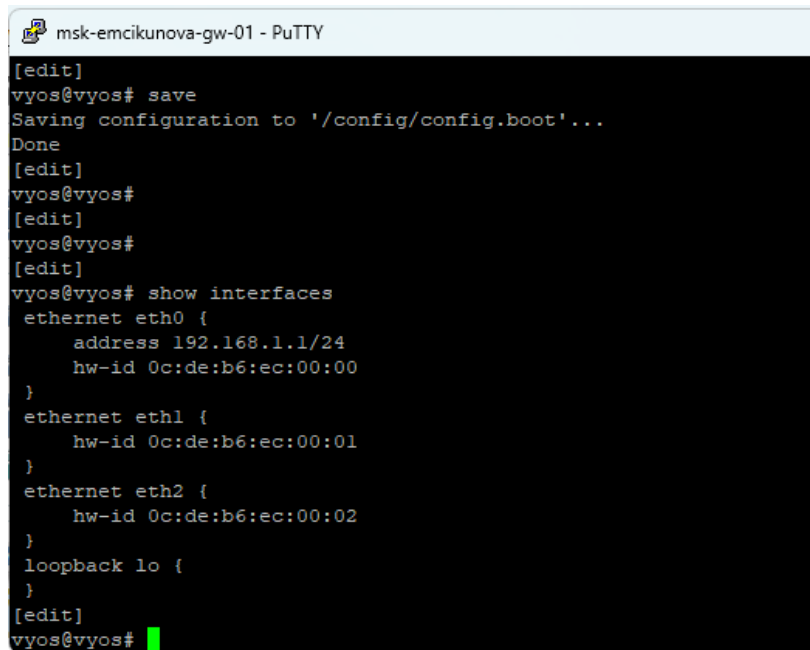
Командой `compare` проверяю внесённые изменения. В выводе отображается удаление параметра `address dhcp`, добавление адреса `192.168.1.1/24` и изменение имени устройства. Применяю конфигурацию командой `commit`, после чего сохраняю её командой `save`. Конфигурация успешно записывается в `/config/config.boot` (см. рис. Рисунок 2.21).



```
msk-emcikunova-gw-01 - PuTTY
VyOS is a free software distribution that includes multiple components,
you can check individual component licenses under /usr/share/doc/*/copyright
vyos@vyos:~$ configure
[edit]
vyos@vyos# set system host-name msk-emcikunova-gw-01
[edit]
vyos@vyos# del interfaces ethernet eth0 address dhcp
[edit]
vyos@vyos# set interfaces ethernet eth0 address 192.168.1.1/24
[edit]
vyos@vyos# compare
[edit interfaces ethernet eth0]
- address dhcp
+ address 192.168.1.1/24
[edit system]
> host-name msk-emcikunova-gw-01
[edit]
vyos@vyos# commit
[edit]
vyos@vyos# save
Saving configuration to '/config/config.boot'...
Done
[edit]
vyos@vyos#
```

Рисунок 2.21: Настройка маршрутизатора VyOS

Проверяю параметры интерфейсов маршрутизатора командой `show interfaces`. Для интерфейса `eth0` отображается настроенный адрес `192.168.1.1/24`. Интерфейсы `eth1` и `eth2` в данной топологии не используются (см. рис. Рисунок 2.22).



```
msk-emcikunova-gw-01 - PuTTY
[edit]
vyos@vyos# save
Saving configuration to '/config/config.boot'...
Done
[edit]
vyos@vyos#
[edit]
vyos@vyos#
[edit]
vyos@vyos# show interfaces
  ethernet eth0 {
    address 192.168.1.1/24
    hw-id 0c:de:b6:ec:00:00
  }
  ethernet eth1 {
    hw-id 0c:de:b6:ec:00:01
  }
  ethernet eth2 {
    hw-id 0c:de:b6:ec:00:02
  }
  loopback lo {
  }
[edit]
vyos@vyos#
```

Рисунок 2.22: Проверка конфигурации интерфейсов маршрутизатора VyOS

С узла PC1-емcikunova выполняю `ping 192.168.1.1`. Маршрутизатор отвечает на все пять ICMP-запросов, TTL ответов равен 64. Полученный результат подтверждает корректность настройки интерфейса VyOS и наличие связи между конечным устройством и маршрутизатором (см. рис. Рисунок 2.23).

```
PC1-emcikunova - PuTTY
VPCS> show ip
NAME       : VPCS[1]
IP/MASK     : 192.168.1.10/24
GATEWAY     : 192.168.1.1
DNS         :
MAC         : 00:50:79:66:68:00
LPORT      : 20004
RHOST:PORT  : 127.0.0.1:20005
MTU         : 1500

VPCS>
VPCS>
VPCS> ping 192.168.1.1

64 bytes from 192.168.1.1 icmp_seq=1 ttl=64 time=3.484 ms
64 bytes from 192.168.1.1 icmp_seq=2 ttl=64 time=4.205 ms
64 bytes from 192.168.1.1 icmp_seq=3 ttl=64 time=2.658 ms
64 bytes from 192.168.1.1 icmp_seq=4 ttl=64 time=2.342 ms
64 bytes from 192.168.1.1 icmp_seq=5 ttl=64 time=2.907 ms

VPCS>
VPCS>
VPCS>
```

Рисунок 2.23: Проверка соединения между PC1 и маршрутизатором VyOS

Анализирую обмен в Wireshark. Перед передачей ICMP-пакетов узел 192.168.1.10 отправляет ARP-запрос Who has 192.168.1.1? Tell 192.168.1.10. Маршрутизатор с адресом 192.168.1.1 отвечает, сообщая свой MAC-адрес 0c:de:b6:ec:00:00. После разрешения MAC-адреса наблюдаются ICMP Echo (ping) request от 192.168.1.10 к 192.168.1.1 и соответствующие Echo (ping) reply от маршрутизатора к конечному устройству. Для каждого запроса присутствует ответ, поэтому соединение функционирует корректно (см. рис. Рисунок 2.24).

No.	Time	Source	Destination	Protocol	Length	Info
22	204.712744	Private_66:68:00	Broadcast	ARP	64	Who has 192.168.1.1? Tell 192.168.1.10
23	204.716558	0c:de:b6:ec:00:00	Private_66:68:00	ARP	60	192.168.1.1 is at 0c:de:b6:ec:00:00
24	204.718204	192.168.1.10	192.168.1.1	ICMP	98	Echo (ping) request id=0xe180, seq=1/256, ttl=64 (reply in 25)
25	204.720700	192.168.1.1	192.168.1.10	ICMP	98	Echo (ping) reply id=0xe180, seq=1/256, ttl=64 (request in 24)
26	205.722606	192.168.1.10	192.168.1.1	ICMP	98	Echo (ping) request id=0xe280, seq=2/512, ttl=64 (reply in 27)
27	205.725427	192.168.1.1	192.168.1.10	ICMP	98	Echo (ping) reply id=0xe280, seq=2/512, ttl=64 (request in 26)
28	206.727967	192.168.1.1	192.168.1.1	ICMP	98	Echo (ping) request id=0xe380, seq=3/768, ttl=64 (reply in 29)
29	206.729627	192.168.1.1	192.168.1.10	ICMP	98	Echo (ping) reply id=0xe380, seq=3/768, ttl=64 (request in 28)
30	207.732318	192.168.1.10	192.168.1.1	ICMP	98	Echo (ping) request id=0xe480, seq=4/1024, ttl=64 (reply in 31)
31	207.733702	192.168.1.1	192.168.1.10	ICMP	98	Echo (ping) reply id=0xe480, seq=4/1024, ttl=64 (request in 30)

[Coloring Rule String: icmp icmpv6] Ethernet II, Src: Private_66:68:00 (00:50:79:66:68:00), Dst: 0c:de:b6:ec:00:00 > Destination: 0c:de:b6:ec:00:00 (0c:de:b6:ec:00:00) > Source: Private_66:68:00 (00:50:79:66:68:00) Type: IPv4 (0x0800) [Stream index: 5] Internet Protocol Version 4, Src: 192.168.1.10, Dst: 192.168.1.1 0100 = Version: 4 0101 = Header Length: 20 bytes (5) > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT) Total Length: 64 Identification: 0x00e2 (32994) > 0000 = Flags: 0x0 ...0 0000 0000 0000 = Fragment Offset: 0 Time to Live: 64 Protocol: ICMP (1) Header Checksum: 0x766b [validation disabled] [Header checksum status: Unverified] Source Address: 192.168.1.10 Destination Address: 192.168.1.1 [Stream index: 1] Internet Control Message Protocol Type: Echo (ping) request (8) Code: 0 Checksum: 0x3d89 [correct] [Checksum Status: Good] Identifier (BE): 57984 (0xe280) Identifier (LE): 32994 (0x00e2) Sequence Number (BE): 2 (0x0002) Sequence Number (LE): 512 (0x0200) [Response frame: 27] > Data (56 bytes)	0000 0c de b6 ec 00 00 00 50 79 66 68 00 08 00 45 00P yfh...E 0010 00 54 80 e2 00 00 40 01 76 6b c0 a8 01 0a c0 a8 -T....@. vk..... 0020 01 01 08 00 3d 89 e2 80 00 02 08 09 0a 0b 0c 0d 0030 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 0040 1e 1f 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d ... !"#%&'()*+,-. 0050 2e 2f 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d ./012345 6789:;<= 0060 3e 3f >?
---	---

Рисунок 2.24: Анализ ARP- и ICMP-трафика между PC1 и маршрутизатором VyOS

После завершения анализа останавливаю захват пакетов в Wireshark, останавливаю все устройства проекта и завершаю работу с GNS3.

3 Вывод

В ходе лабораторной работы были построены простейшие сети в GNS3 на базе коммутатора Ethernet и маршрутизаторов FRR и VyOS. Выполнена настройка IP-адресации и проверена связность между сетевыми устройствами. С помощью Wireshark был проанализирован трафик протоколов ARP, ICMP, UDP и TCP. Все созданные соединения работали корректно, отправленные пакеты успешно достигали адресатов.